CSC 316

Handout: Svelte 5 Runes & Reactivity

What this is. The one idea that makes an interactive dashboard work: you *describe* what each value depends on, and Svelte keeps everything in sync for you. **Companion to Lab 4**, where you connect several views to a shared selection. The optional selector in Lab 2 is an early preview.

The examples use a **LAX dashboard** with a terminal dropdown that filters a line chart and three summary cards. This is a reference example; Lab 2 uses two static sketches, and Lab 4 introduces coordinated interaction with a different dataset. You don't need to write much of this from scratch — the goal is to **read** the code Claude wrote and be able to point at each line and say what it does. That is exactly process-book **Question 6**.

---

1. The mental shift

Most programming you've seen is **imperative**: you give an ordered list of steps, and when something changes you write more steps to update everything by hand.

```
// imperative — the old way (NOT how we work)
dropdown.addEventListener('change', () => {
  const rows = filterRows(dropdown.value);   // 1. refilter
  updateLineChart(rows);                      // 2. redraw the chart
  updateCard1(rows);                          // 3. redraw card 1
  updateCard2(rows);                          // 4. redraw card 2
  updateCard3(rows);                          // 5. ...and don't forget any!
});
```

The problem: **you** are responsible for remembering every view that depends on the selection. Forget one, and your cards disagree with your chart — a classic "lying dashboard."

Svelte is **declarative and reactive**. You don't write "when X changes, update Y." You write "**Y is a function of X**," once, and Svelte re-runs it whenever X changes:

```
<script>
  let selectedTerminal = $state('All terminals');          // X: the user changes this
  let filtered = $derived(rowsFor(selectedTerminal));      // Y: defined once as a function of X
</script>
```

Change `selectedTerminal` and `filtered` recomputes on its own; anything drawn from `filtered` re-renders on its own. You never call an "update" function. **That shift — from *doing updates* to *describing dependencies* — is the whole reason we use Svelte.**

> **A rune is a compiler hint.** `$state`, `$derived`, `$props`, and `$effect` are **runes** — special markers (note the `$`) that tell the Svelte compiler "this value is reactive; track it." They aren't normal function calls, and they only work inside `.svelte` files (and `.svelte.js` files). You'll never `import` them.

---

2. `$state` — the things the user changes

`$state(...)` marks a value as **reactive state**: the source-of-truth values a user can change. In the dashboard there's really just one — which terminal is selected:

```
<script>
  let selectedTerminal = $state('All terminals');
</script>
```

`selectedTerminal` now behaves like a normal variable — you can read it, and reassigning it (`selectedTerminal = 'T5'`) is what kicks off every downstream update.

Wiring it to the dropdown. The `<select>` element reads and writes that state with `bind:value` — a two-way connection, so picking an option *is* the reassignment:

```
<select bind:value={selectedTerminal}>
  <option>All terminals</option>
  {#each terminals as t}
    <option>{t}</option>
  {/each}
</select>
```

In your project the dropdown is its own `TerminalFilter.svelte` component; the parent's state is handed down with a `$bindable` prop so the child can drive it. That's a refinement — the idea is identical: **one `$state` value, changed in one place.**

> **State should be minimal.** Store only what the user actually controls (the selection). *Don't* store the filtered rows or the totals in `$state` — those are computed, and computed values belong to `$derived` (next). A good rule: if you could recalculate it, it isn't state.

---

3. `$derived` — everything computed from state

`$derived(...)` declares a value that is **computed from other reactive values and recomputes automatically** when any of them change. This is where the actual work of the dashboard lives.

```
<script>
  // the filtered rows: a function of the selection
  let filtered = $derived(
    selectedTerminal === 'All terminals'
      ? rows
      : rows.filter(d => d.Terminal === selectedTerminal)
  );
</script>
```

You never assign to `filtered` yourself and you never "refresh" it. Svelte notices that the expression **reads** `selectedTerminal`, so whenever that changes, `filtered` recomputes.

Deriving from a derived value is fine — encouraged, even. The three KPI cards each derive from `filtered`:

```
<script>
  let totalPassengers = $derived(
    filtered.reduce((sum, d) => sum + d.Passenger_Count, 0)
  );
</script>
```

Change the dropdown → `selectedTerminal` changes → `filtered` recomputes → `totalPassengers` recomputes → the card re-renders. A **chain of dependencies**, each link declared once.

When the computation needs several statements, use `$derived.by(() => { ... })` — same idea, but you get a full function body and `return` the result:

```
<script>
  let byMonth = $derived.by(() => {
    const grouped = d3.rollup(
      filtered,
      v => d3.sum(v, d => d.Passenger_Count),
      d => d.month
    );
    return [...grouped].sort((a, b) => a[0] - b[0]);   // the line chart's data
  });
</script>
```

> **The rule of thumb.** `$state` = a handful of values the user changes. `$derived` = *everything else* — filtered data, aggregates, scales, chart data. If a value can be computed from other values, make it `$derived` and never touch it again.

---

4. How a change propagates

Here's the full chain when you pick **T5** from the dropdown. This is the "why does the whole page update?" answer:

```
  you pick "T5"
        │
        ▼
  selectedTerminal  ($state)          ← the ONE thing that actually changed
        │
        ▼
  filtered  ($derived)                ← recomputes: rows where Terminal === "T5"
        │
        ├──────────────┬──────────────┐
        ▼              ▼              ▼
  totalPassengers  byMonth        (other cards)   ($derived, all recompute)
        │              │              │
        ▼              ▼              ▼
    KpiCard        LineChart        KpiCards       ← every view that reads them re-renders
```

You wrote **one** assignment (`selectedTerminal = 'T5'` , via the dropdown). Svelte did the rest by following the dependencies you declared. There is **no hidden "update everything" call** — and because every view reads from the same derived data, they physically *cannot* disagree. This is the shared-data rule you examine in Lab 4.

> **Mapping to process-book Q6.** This is the code to label *data in* → *transformation* → *output*: **- data in** — the `d3.csv` load that fills `rows` (see D3, the Way We Use It). **- transformation** — the `$state` selection and the `$derived` values built from it (this handout). **- output** — the components (`KpiCard` , `LineChart`) that render the derived data (see Svelte Components).

5. `$props` and `$effect` (the other two you'll meet)

`$props()` — how a component receives data. A component like `KpiCard` doesn't own its numbers; the page passes them in:

```
<!-- KpiCard.svelte -->
<script>
  let { label, value } = $props();     // receive props from the parent
</script>
```

```
<!-- +page.svelte uses it three times, reusing ONE component -->
<KpiCard label="Total passengers"   value={totalPassengers} />
<KpiCard label="Busiest terminal"   value={busiest} />
<KpiCard label="Recovered vs. 2019" value={pctRecovered} />
```

The reactivity point — and the only part that belongs in *this* handout — is that **props stay reactive across the component boundary.** `totalPassengers` is a `$derived` value; the card re-renders when the selection changes, without the card knowing anything about the dropdown. Everything else about props (defaults, renaming, string-vs-expression, how small a component should be) is in Svelte Components §2.

`$effect(() => { ... })` — for genuine side effects only. An effect runs *after* the DOM updates and re-runs when the reactive values it reads change. Use it for things that reach *outside* Svelte's data flow — logging, saving to `localStorage`, drawing to a `<canvas>`:

```
<script>
  $effect(() => {
    console.log('now showing', selectedTerminal);   // re-runs whenever the selection changes
  });
</script>
```

The one `$effect` you *will* write: axes. D3's axis generators build their own DOM, so the `<g>` holding an axis is genuinely outside Svelte's data flow — which makes it the course's standard, sanctioned effect:

```
<g bind:this={axisEl} transform="translate(0,{innerH})" />
```

```
$effect(() => { d3.select(axisEl).call(d3.axisBottom(xScale)); });
```

It re-runs whenever `xScale` changes, so the axis stays in sync with a filtered domain. See D3, the Way We Use It §4 for the full pattern.

> **Outside those two cases you almost never need `$effect`.** The most common beginner mistake is using an effect to *compute* a value ("when the selection changes, set `filtered = ...`"). Don't — that's what `$derived` is for, and `$derived` is simpler, faster, and can't get out of sync. Reach for `$effect` only when you're talking to something *outside* Svelte, as the axis is.

6. Common pitfalls

Pitfall	What happens	Fix
Using `$effect` to set a value from other state	Extra re-renders, values that lag or desync	Use `$derived` — it's the right tool for "computed from"

Pitfall	What happens	Fix
Storing computed data in `$state`	It goes stale; you end up manually re-setting it	Store only what the user changes; derive the rest
Assigning to a `$derived` value	It's read-only; Svelte owns it	Change the underlying `$state` instead
Mutating a **plain** (non-`$state`) array/object	The view doesn't update	Make it `$state` — Svelte 5 state is deeply reactive, so `arr.push(...)` *does* trigger updates
Destructuring reactive values too early	You capture a snapshot and lose reactivity	Keep reading through the variable (`data.value`), don't pull the value out into a plain `const` at the top

> **The deep-reactivity upgrade (Svelte 5).** Objects and arrays wrapped in `$state` are *deeply* reactive — mutating them in place (`push`, `splice`, setting a property) triggers updates, no reassignment needed. This is new in Svelte 5; older Svelte code you find online reassigns arrays to force updates, which you no longer have to do.

Appendix — The reactive dashboard, annotated

The reactive core of `src/routes/+page.svelte`, with every reactive line labeled. This is the exact code to read for process-book **Question 6**:

```
<script>
  import { onMount } from 'svelte';
  import * as d3 from 'd3';
  import KpiCard from '$lib/KpiCard.svelte';
  import LineChart from '$lib/LineChart.svelte';
  import TerminalFilter from '$lib/TerminalFilter.svelte';

  // ── data in ──────────────────────────────────────────────
  let rows = $state([]);                          // filled once the CSV loads
  onMount(async () => {
    const raw = await d3.csv('/lax_passenger_traffic.csv');
    rows = raw.map(d => ({                         // parse each row
      Terminal: d.Terminal,
      month: d3.timeParse('%m/%d/%Y %I:%M:%S %p')(d.ReportPeriod),
      Passenger_Count: +d.Passenger_Count          // string → number
    }));
  });

  // ── the ONE piece of state the user controls ─────────────
  let selectedTerminal = $state('All terminals');

  // ── transformation: everything below is DERIVED ──────────
  let terminals = $derived([...new Set(rows.map(d => d.Terminal))].sort());
  let filtered  = $derived(
    selectedTerminal === 'All terminals'
      ? rows
      : rows.filter(d => d.Terminal === selectedTerminal)
  );
  let totalPassengers = $derived(filtered.reduce((s, d) => s + d.Passenger_Count, 0));
  let byMonth = $derived.by(() => {
    const g = d3.rollup(filtered, v => d3.sum(v, d => d.Passenger_Count), d => d.month);
    return [...g].sort((a, b) => a[0] - b[0]);
  });
</script>

<!-- ── output: every view reads the derived values ──────── -->
<TerminalFilter {terminals} bind:selected={selectedTerminal} />

<KpiCard label="Total passengers" value={totalPassengers} />
<LineChart data={byMonth} />

<p class="caption">
  "All terminals" sums arrivals + departures × domestic + international.
</p>
```

Read it top to bottom: **data in** (load + parse), **transformation** (one `$state`, several `$derived`), **output** (components bound to the derived values). Change the one `$state` value and the whole page follows — that's reactivity, and that's the entire model.

Quick reference

Rune	Means	In the dashboard
`$state(x)`	a value the user changes	`selectedTerminal`
`$derived(expr)`	computed, auto-updating	`filtered`, `totalPassengers`
`$derived.by(() => …)`	same, multi-statement	`byMonth`
`$props()`	data a component receives	`KpiCard`'s `label`, `value`
`$effect(() => …)`	a side effect outside Svelte	drawing a D3 axis; logging